Lab: Debugging and Troubleshooting an SSIS Package

Scenario

The ETL process for Adventure Works Cycles occasionally fails when extracting data from text files generated by the company's financial accounts system. You plan to debug the ETL process to identify the source of the problem and implement a solution to handle any errors.

Objectives

After completing this lab, you will be able to:

- Debug an SSIS package.
- Log SSIS package execution.
- Implement an event handler in an SSIS package.
- Handle errors in a data flow.

Estimated Time: 60 minutes

Virtual machine: **20767C-MIA-SQL**

User name: **ADVENTUREWORKS\Student**

Password: **Pa55w.rd**

Exercise 1: Debugging an SSIS Package

Scenario

You have developed an SSIS package to extract data from text files exported from a financial accounts system and load the data into a staging database. However, while developing the package, you have encountered some errors and you need to debug it to identify the cause.

The main tasks for this exercise are as follows:

1. Prepare the Lab Environment
2. Run an SSIS Package
3. Add a Breakpoint
4. Add a Data Viewer
5. View Breakpoints
6. Observe Variables while Debugging
7. View Data Copied from a Data Viewer

▶ Task 1: Prepare the Lab Environment

1. Ensure the **20767C-MIA-DC** and **20767C-MIA-SQL** virtual machines are both running, and then log on to **20767C-MIA-SQL** as **ADVENTUREWORKS\Student** with the password **Pa55w.rd**
2. Run **Setup.cmd** in the **D:\Labfiles\Lab08\Starter** folder as Administrator.

▶ Task 2: Run an SSIS Package

1. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab08\Starter\Ex1** folder.
2. Open the **Extract Payment Data.dtsx** package and examine its control flow.
3. Run the package, noting that it fails. When execution is complete, stop debugging.

▶ Task 3: Add a Breakpoint

1. Add a breakpoint to the **Extract Payments** data flow task in the Extract Payment Data.dtsx package.
2. Select the appropriate event to ensure that execution is always paused at the beginning of every iteration of the loop.

▶ Task 4: Add a Data Viewer

- In the data flow for the **Extract Payments** task, add a data viewer to the data flow path between the **Payments File** source and the **Staging DB** destination.

▶ Task 5: View Breakpoints

- View the Breakpoints window, and note that it contains the breakpoint you defined on the Foreach Loop container and the data viewer you added to the data flow.

▶ Task 6: Observe Variables while Debugging

1. Start debugging the package, and note that execution pauses at the first breakpoint.
2. View the Locals window and view the configuration values and variables it contains.
3. Add the following variables to the Watch 1 window:
   - User::fName
   - $Project::AccountsFolderPath
4. Continue execution and note that it pauses at the next breakpoint, which is the data viewer you defined in the data flow. Also note that the data viewer window contains the contents of the file indicated by the **User::fName** variable in the Watch 1 window.
5. Continue execution, observing the value of the **User::fName** variable and the contents of the data viewer window during each iteration of the loop.
6. When the **Extract Payments** task fails, note the value of the **User::fName** variable, and copy the contents of the data viewer window to the clipboard. Then stop debugging and close Visual Studio.

▶ Task 7: View Data Copied from a Data Viewer

1. Start Notepad and paste the data you copied from the data viewer window into a worksheet.
2. Examine the data and try to determine the errors it contains (Hint: look at payments 4074, 4102, and 4124).

Results: After this exercise, you should have observed the variable values and data flows for each iteration of the loop in the Extract Payment Data.dtsx package. You should also have identified the file that caused the data flow to fail and examined its contents to find the data errors that triggered the failure.

Exercise 2: Logging SSIS Package Execution

Scenario

You have debugged the Extract Payment Data.dtsx package and found some errors in the source data. Now you want to implement logging to assist in diagnosing future errors when the package is deployed in a production environment.

The main tasks for this exercise are as follows:

1. Configure SSIS Logs

2. View Logged Events

▶ Task 1: Configure SSIS Logs

1. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab08\Starter\Ex2** folder.
2. Open the **Extract Payment Data.dtsx** package and configure logging with the following settings:
   - Enable logging for the package, and inherit logging settings for all child executables.
   - Log package execution events to the Windows Event Log.
   - Log all available details for the OnError and OnTaskFailed events.

▶ Task 2: View Logged Events

1. Run the package in Visual Studio, and note that it fails. When execution is complete, stop debugging.
2. View the Log Events window and note the logged events it contains. If no events are logged, rerun the package and look again, then close Visual Studio.
3. View the Application window event log in the Event Viewer administrative tool.

Results: After this exercise, you should have a package that logs event details to the Windows Event Log.

Exercise 3: Implementing an Event Handler

Scenario

You have debugged the Extract Payment Data.dtsx package and observed how errors in the source data can cause it to fail. You now want to implement an error handler that copies the invalid source data file to a folder for later examination and notifies an administrator.

The main tasks for this exercise are as follows:

1. Create an Event Handler

2. Add a File System Task

3. Add a Send Mail Task

4. Test the Event Handler

▶ Task 1: Create an Event Handler

1. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab08\Starter\Ex3** folder.
2. In the **Extract Payment Data.dtsx** package, create an event handler for the **OnError** event of the **Extract Payment Data** executable.

▶ Task 2: Add a File System Task

1. Add a file system task to the control flow for the OnError event handler, and name it **Copy Failed File**.
2. Configure the Copy Failed File task as follows:
   - The task should perform a **Copy file** operation.
   - Use the **Payments File** connection manager as the source connection.
   - Create a new connection manager to create a file named **D:\ETL\FailedPayments.csv** for the destination.
   - The task should overwrite the destination file if it already exists.
3. Configure the connection manager you created for the destination file to use the following expression for its **ConnectionString** property. Instead of the name configured in the connection manager, the copied file is named using a combination of the unique package execution ID and the name of the source file:

```
"D:\\ETL\\" + @[System::ExecutionInstanceGUID] + @[User::fName]
```

▶ Task 3: Add a Send Mail Task

1. Add a send mail task to the control flow for the OnError event handler, and name it **Send Notification**.
2. Connect the **Copy Failed File** task to the **Send Notification** task with a **Completion precedence** constraint.
3. Configure the **Notification** task as follows:
   - Use the **Local SMTP Server** connection manager.
   - Send a high-priority email message from etl@adventureworks.msft to student@adventureworks.msft with the subject "An error occurred".
   - Use the following expression to set the **MessageSource** property:

```
@[User::fName] + " failed to load. " +
@[System::ErrorDescripti
on]
```

▶ Task 4: Test the Event Handler

1. Run the package in Visual Studio and verify that the event handler is executed. Close Visual Studio, saving your changes, if prompted.
2. Verify that the source file containing invalid data is copied to the **D:\ETL** folder with a name similar to {1234ABCD-1234-ABCD-1234-ABCD1234}Payments - EU.csv.
3. View the contents of the **C:\inetpub\mailroot\Drop** folder, and verify that an email was sent for each error that occurred. You can examine each email file by using Notepad.

Results: After this exercise, you should have a package that includes an event handler for the OnError event. The event handler should create a copy of files that contain invalid data and send an email message.

Exercise 4: Handling Errors in a Data Flow

Scenario

You have implemented an error handler that notifies an operator when a data flow fails. However, you would like to handle errors in the data flow so that only rows containing invalid data are not loaded, and the rest of the data flow succeeds.

The main tasks for this exercise are as follows:

1. Redirect Data Flow Errors

2. View Invalid Data Flow Rows

▶ Task 1: Redirect Data Flow Errors

1. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab08\Starter\Ex4** folder.
2. Open the **Extract Payment Data.dtsx** package and view the data flow for the **Extract Payments** task.
3. Configure the error output of the **Staging DB** destination to redirect rows that contain an error.
4. Add a flat file destination to the data flow and name it **Invalid Rows**. Then configure the **Invalid Rows** destination as follows:
    - Create a new connection manager named **Invalid Payment Records** for a delimited file named **D:\ETL\InvalidPaymentsLog.csv**.
    - Do not overwrite data in the text file if it already exists.
    - Map all columns, including **ErrorCode** and **ErrorColumn**, to fields in the text file.

▶ Task 2: View Invalid Data Flow Rows

1. Run the package in debug mode and note that it succeeds. When execution is complete, stop debugging and close Visual Studio.
2. Use Notepad to view the contents of the InvalidPaymentsLog.csv file in the **D:\ETL** folder and note the rows that contain invalid values.

Results: After this exercise, you should have a package that includes a data flow where rows containing errors are redirected to a text file.